

Web Service Design and Practice

05. Node.js Basics

Dr. Kyudong Park

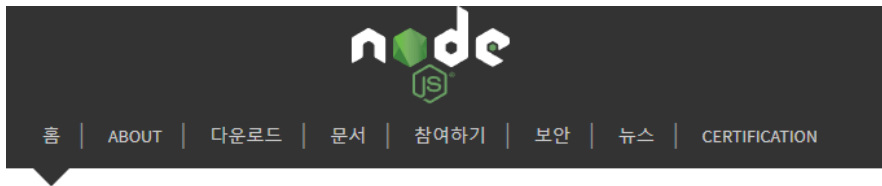
School of Information Convergence



Contents

- [NodeJS 소개](#)
- 호출 스택과 이벤트 루프
- 요청과 응답
- REST API

Node의 정의

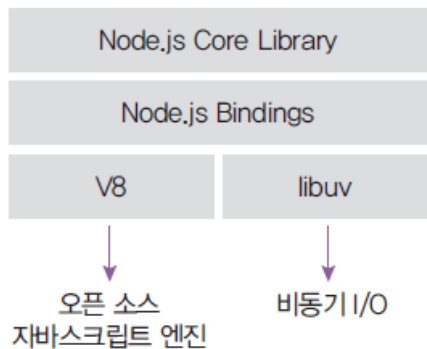


Node.js®는 Chrome V8 JavaScript 엔진으로 빌드된 JavaScript 런타임입니다.

- 노드는 서버가 아닌가요?
 - 서버의 역할도 수행할 수 있는 자바스크립트 런타임
 - 런타임: 특정 언어로 만든 프로그램들을 실행할 수 있게 해주는 가상 머신(크롬의 V8 엔진 사용)의 상태
 - 다른 런타임으로는 웹 브라우저(크롬, 엣지, 사파리, 파이어폭스 등)가 있음
 - 노드로 자바스크립트로 작성된 프로그램(서버)을 실행할 수 있음.
 - 서버 실행을 위해 필요한 http/https/http2 모듈을 제공

Node의 내부구조

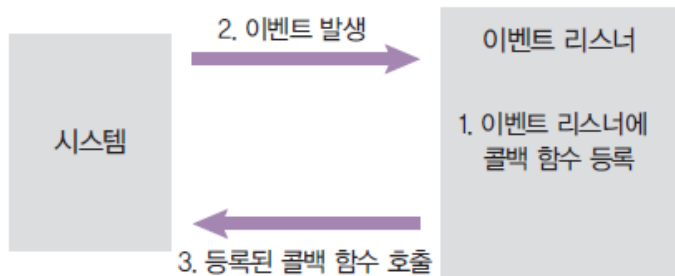
- 2008년 V8 엔진 출시, 2009년 노드 프로젝트 시작
- 노드는 V8과 libuv를 내부적으로 포함
 - V8 엔진: 오픈 소스 자바스크립트 엔진 -> 속도 문제 개선
 - libuv: 노드의 특성인 이벤트 기반, 논블로킹 I/O 모델을 구현한 라이브러리



Node의 특성 1: 이벤트 기반

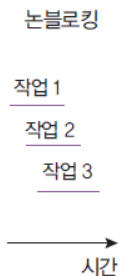
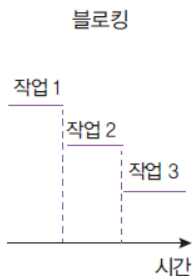
- 이벤트가 발생할 때 미리 지정해둔 작업을 수행하는 방식
 - 이벤트의 예: 클릭, 네트워크 요청, 타이머 등
 - 이벤트 리스너: 이벤트를 등록하는 함수
 - 콜백 함수: 이벤트가 발생했을 때 실행될 함수

▼ 그림 1-4 이벤트 기반



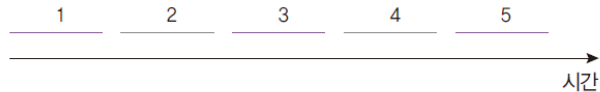
Node의 특성 2: Non-Blocking I/O

- 논 블로킹: 오래 걸리는 함수를 백그라운드로 보내서 다음 코드가 먼저 실행되게 하고, 나중에 오래 걸리는 함수를 실행
 - 논 블로킹 방식 하에서 일부 코드는 백그라운드에서 병렬로 실행됨
 - 일부 코드: I/O 작업(파일 시스템 접근, 네트워크 요청), 압축, 암호화 등
 - I/O작업을 제외한 나머지 코드는 블로킹 방식으로 실행됨
 - ∴ I/O 작업이 많을 때 노드 활용성이 극대화

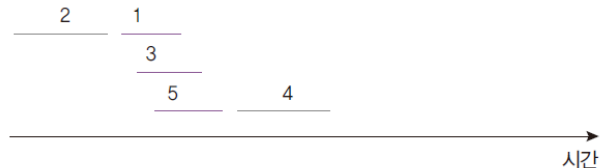


▼ 그림 1~10 동시 처리로 얻는 시간적 이득

case 1



case 2



—— 동시 처리 가능
—— 동시 처리 불가능

Node의 역할

- 서버로서의 node
 - 서버: 네트워크를 통해 클라이언트에 정보나 서비스를 제공하는 컴퓨터 또는 프로그램
 - 클라이언트: 서버에 요청을 보내는 주체(브라우저, 데스크탑 프로그램, 모바일 앱, 다른 서버에 요청을 보내는 서버)
- 예시
 - 브라우저(클라이언트, 요청)가 특정 웹사이트(서버, 응답)에 접속
 - 핸드폰(클라이언트)을 통해 앱스토어(서버)에서 앱 다운로드
- 노드 != 서버
- But, 노드는 서버를 구성할 수 있게 하는 모듈을 제공

Node의 역할

- node 서버의 장단점

장점	단점
멀티 스레드 방식에 비해 컴퓨터 자원을 적게 사용함	싱글 스레드라서 CPU 코어를 하나만 사용함
I/O 작업이 많은 서버로 적합	CPU 작업이 많은 서버로는 부적합
멀티 스레드 방식보다 쉬움	하나뿐인 스레드가 멈추지 않도록 관리해야 함
웹 서버가 내장되어 있음	서버 규모가 커졌을 때 서버를 관리하기 어려움
자바스크립트를 사용함	어중간한 성능
JSON 형식과 호환하기 쉬움	

- 페이팔, 넷플릭스, 나사, 월마트, 링크드인, 우버 등에서 메인 또는 서브 서버로 사용

Node의 역할

- 서버 외의 활용
 - 자바스크립트 런타임이기 때문에 용도가 서버에만 한정되지 않음
 - 웹, 모바일, 데스크탑 애플리케이션에도 사용
 - 웹 프레임워크: Angular, React, Vue, Meteor 등
 - 모바일 앱 프레임워크: React Native
 - 데스크탑 개발 도구: Electron(Atom, Slack, VSCode, Discord 등 제작)

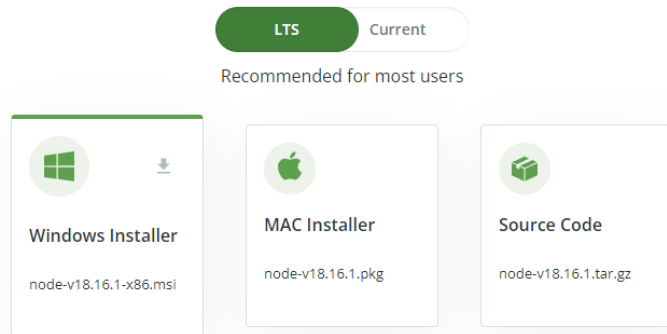
Node 설치

- <https://nodejs.dev/en/download/>
- LTS 버전 권장
 - Long-term support
- 설치 제대로 되었는지 확인
 - `$> node --version`

HOME / DOWNLOADS Downloads

LATEST LTS VERSION V18.16.1
(includes npm)

Download the Node.js source code, a pre-built installer for your platform, or install via a [package manager](#).



npm

- Node Package Manager

- 노드의 패키지 매니저
- 다른 사람들이 만든 소스 코드들을 모아둔 저장소
- 남의 코드를 사용하여 프로그래밍 가능
- 이미 있는 기능을 다시 구현할 필요가 없어 효율적
- 오픈 소스 생태계를 구성 중

- 패키지: npm에 업로드된 노드 모듈
- 모듈이 다른 모듈을 사용할 수 있듯 패키지도 다른 패키지를 사용할 수 있음
- 의존 관계라고 부름



package.json

- 현재 프로젝트에 대한 정보와 사용 중인 패키지에 대한 정보를 담은 파일
 - 같은 패키지라도 버전별로 기능이 다를 수 있으므로 버전을 기록해두어야 함
 - 동일한 버전을 설치하지 않으면 문제가 생길 수 있음
 - 노드 프로젝트 시작 전 package.json부터 만들고 시작함(npm init)

콘솔

```
$ npm init
```

```
This utility will walk you through creating a package.json file.
```

```
It only covers the most common items, and tries to guess sensible defaults.
```

```
See `npm help json` for definitive documentation on these fields  
and exactly what they do.
```

- npm init이 완료되면 폴더에 package.json이 생성됨
- npm run [스크립트명]으로 스크립트 실행

package.json

- express 설치하기

콘솔

```
$ npm install express
npm notice created a lockfile as package-lock.json. You should commit this file.
npm WARN npmtest@0.0.1 No repository field.

+ express@4.17.1
added 285 packages from 163 contributors and audited 2401 packages in 6.314s
```

- package.json 에 기록 확인

package.json

```
{
  "name": "npmtest",
  ...
  "license": "ISC",
  "dependencies": {
    "express": "^4.17.1",
```

node_modules

- npm install 시 node_modules 폴더 생성
 - 내부에 설치한 패키지들이 들어 있음
 - express 외에도 express와 의존 관계가 있는 패키지들이 모두 설치됨
- package-lock.json도 생성되어 패키지 간 의존 관계를 명확하게 표시함

VS Code 에서 npm 안될때

- Windows PowerShell 의 권한 문제

```
PS C:\Users\kdpark\Documents\GitHub\KW2021_WebProgramming> nodemon
nodemon : 이 시스템에서 스크립트를 실행할 수 없으므로 C:\Users\kdpark\AppData\Roaming\npm\nodemon.ps1 파일을 로드할 수 없습니다. 자세한 내용은 about_Execution_Policies(https://go.microsoft.com/fwlink/?LinkID=135170)를 참조하십시오.
```

```
PS C:\Windows\system32> Set-ExecutionPolicy RemoteSigned
```

개발용 패키지

- `npm install --save-dev 패키지명` 또는 `npm i -D 패키지명`
 - `devDependencies`에 추가됨
 - 개발 환경에서만 설치

콘솔

```
$ npm install --save-dev nodemon  
+ nodemon@1.17.3  
added 227 packages from 134 contributors in 24.735s
```

package.json

```
{  
  ...  
  "devDependencies": {  
    "nodemon": "^2.0.3"  
  }  
}
```


글로벌 (전역) 패키지

- `npm install --global` 패키지명 또는 `npm i -g` 패키지명
 - 모든 프로젝트와 콘솔에서 패키지를 사용할 수 있음
 - 예제는 `rm -rf`(리눅스의 삭제 명령)를 흉내내는 `rimraf` 패키지의 글로벌 설치
 - `npx`로 설치 없이 사용 가능
 - `node package execute` (로컬이나 글로벌로 설치하지 않고도, 패키지를 임시로 다운로드하고 실행)

콘솔

```
$ npm install --global rimraf
C:\Users\zerocho\AppData\Roaming\npm\rimraf -> C:\Users\zerocho\AppData\Roaming\npm\node_modules\rimraf\bin.js

+ rimraf@3.0.2
added 12 packages from 4 contributors in 1.015s
```

콘솔

```
$ npm install --save-dev rimraf
$ npx rimraf node_modules
```

package.json

- 결국 중요한 것은 package.json
 - npm install 만으로 자동 설치
- 프로젝트 공유 시 node_module 은 굳이 동봉할 필요 없음
 - git 에서 node_module 은 제외 (ignore) 되어있음

NVM (node version manager)

- Node 의 버전을 변경하면서 개발할 때 활용
- 원하는 node 버전으로의 변경 / 설치 가능
- Windows
 - <https://github.com/coreybutler/nvm-windows>
- Linux / MacOS
 - <https://github.com/nvm-sh/nvm>

Contents

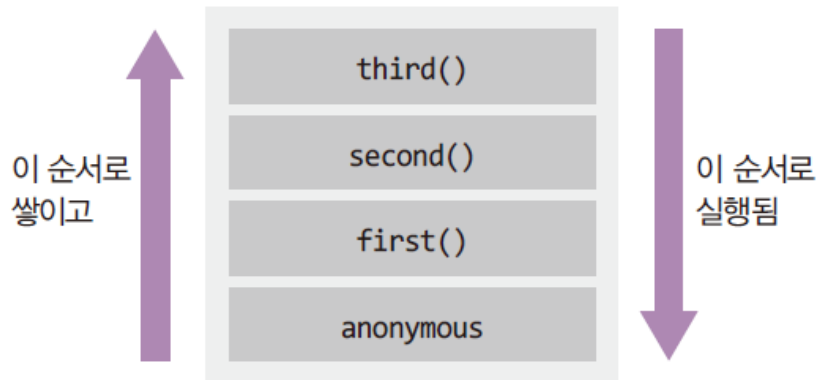
- NodeJS 소개
- 호출 스택과 이벤트 루프
- 요청과 응답
- REST API

호출 스택과 이벤트 루프

- 호출 스택
 - 코드 순서 예측해보기

```
function first() {  
  second();  
  console.log('첫 번째');  
}  
  
function second() {  
  third();  
  console.log('두 번째');  
}  
  
function third() {  
  console.log('세 번째');  
}  
  
first();
```

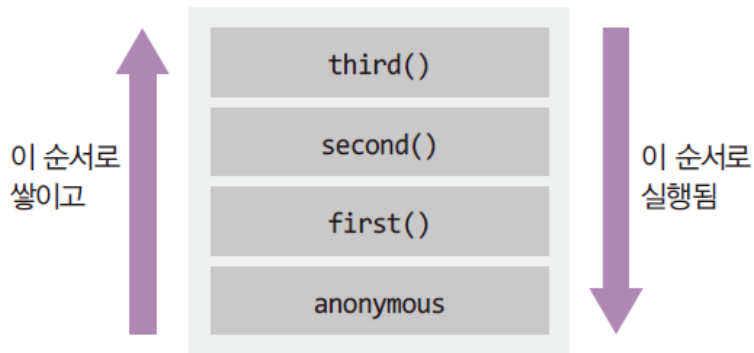
▼ 그림 1-5 호출 스택



호출 스택과 이벤트 루프

- 호출 스택
 - 함수의 호출, 자료구조의 Stack
 - Anonymous은 가상의 전역 컨텍스트
 - 함수 호출 순서대로 쌓이고, 역순으로 실행됨
 - 함수 실행이 완료되면 스택에서 빠짐
 - LIFO 구조라서 스택이라고 불림

▼ 그림 1-5 호출 스택



호출 스택과 이벤트 루프

- 이벤트루프의 필요

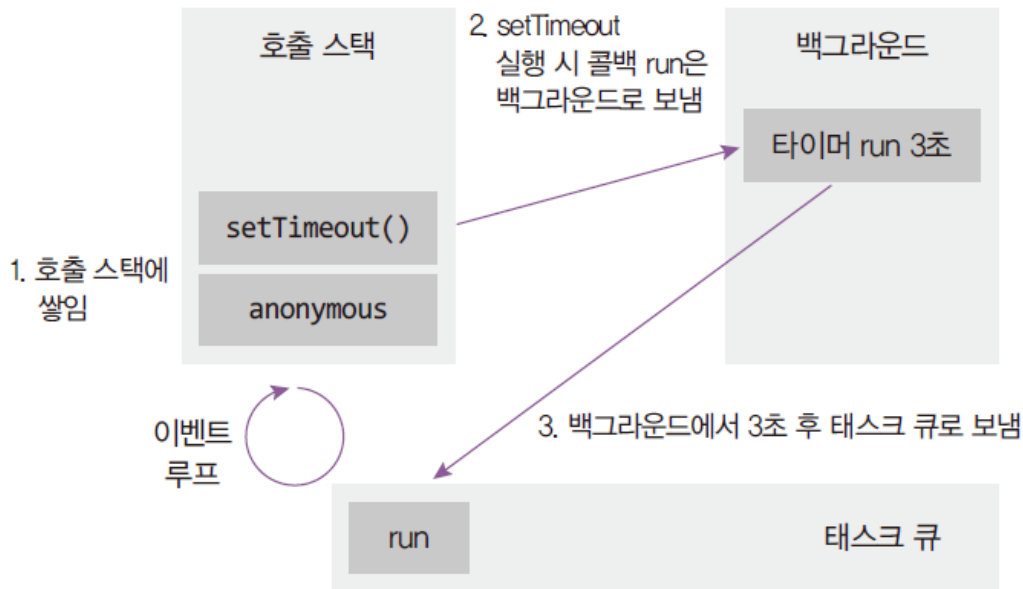
- 아래 코드의 실행 순서

```
function run() {  
  console.log('3초 후 실행');  
}  
console.log('시작');  
setTimeout(run, 3000);  
console.log('끝');
```

- 호출 스택만으로는 설명 되지 않음

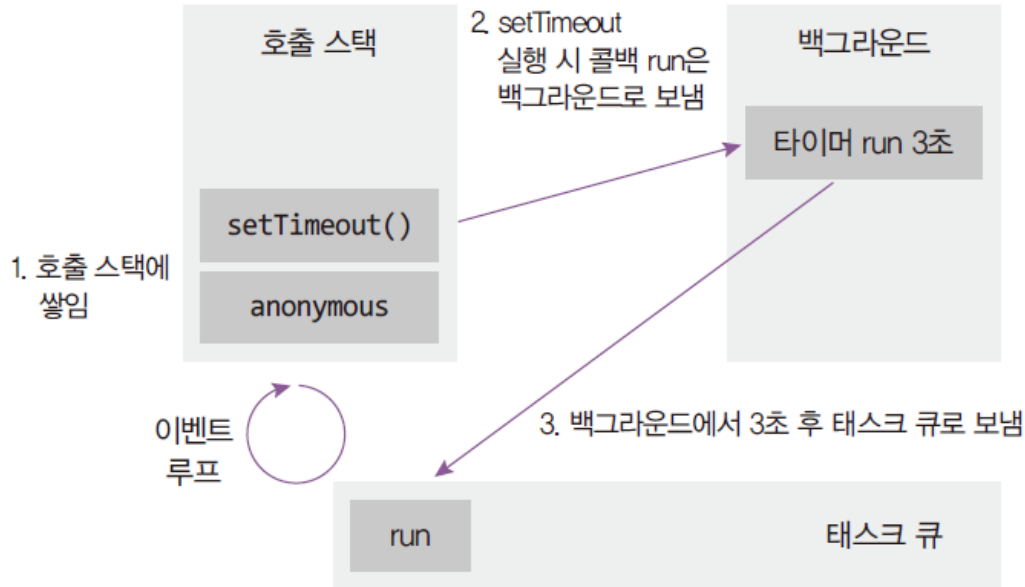
호출 스택과 이벤트 루프

- 이벤트 루프: 이벤트 발생(setTimeout 등) 시 호출할 콜백 함수들을 관리하고, 호출할 순서를 결정하는 역할
- 태스크 큐: 이벤트 발생 후 호출되어야 할 콜백 함수들이 순서대로 기다리는 공간
- 백그라운드: 타이머나 I/O 작업 콜백, 이벤트 리스너들이 대기하는 공간. 여러 작업이 동시에 실행될 수 있음



호출 스택과 이벤트 루프

- 예제 코드에서 `setTimeout`이 호출될 때 콜백 함수 `run`은 백그라운드로
 - 백그라운드에서 3초를 보냄
 - 3초가 다 지난 후 백그라운드에서 태스크 큐로 보내짐



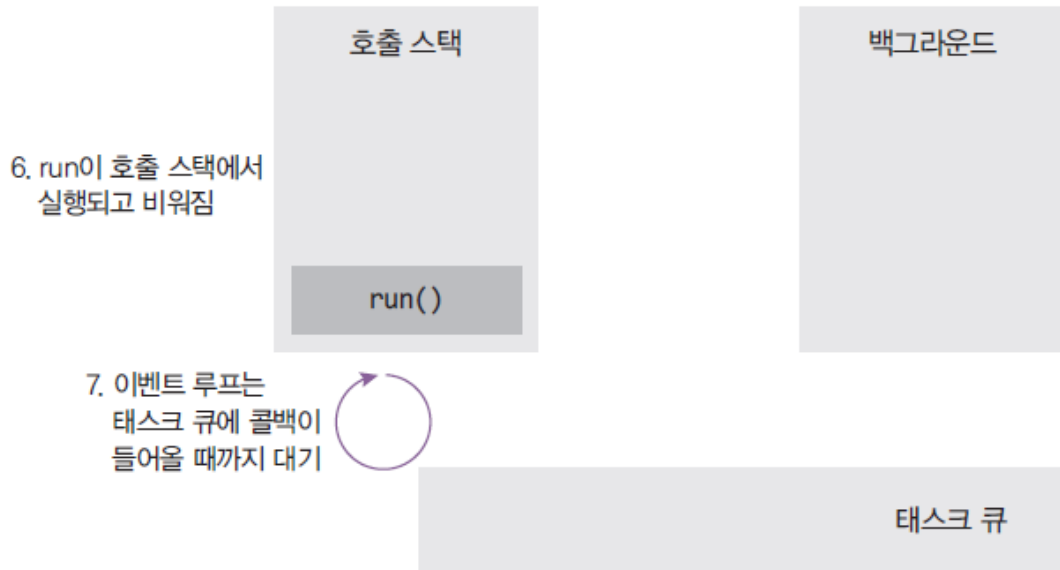
호출 스택과 이벤트 루프

- setTimeout과 anonymous가 실행 완료된 후 호출 스택이 완전히 비워지면, 이벤트 루프가 태스크 큐의 콜백을 호출 스택으로 올림
 - 호출 스택이 비워져야만 올림
 - 호출 스택에 함수가 많이 차 있으면 그것들을 처리하느라 3초가 지난 후에도 run 함수가 태스크 큐에서 대기하게 됨 -> 타이머가 정확하지 않을 수 있는 이유



호출 스택과 이벤트 루프

- run이 호출 스택에서 실행되고, 완료 후 호출 스택에서 나감
 - 이벤트 루프는 task 큐에 다음 함수가 들어올 때까지 계속 대기
 - task 큐는 실제로 여러 개고, task 큐들과 함수들 간의 순서를 이벤트 루프가 결정함

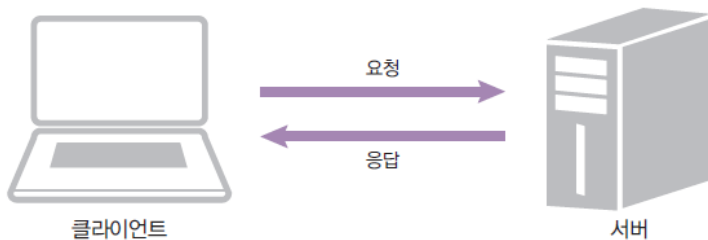


Contents

- NodeJS 소개
- 호출 스택과 이벤트 루프
- [요청과 응답](#)
- REST API

요청과 응답

- request & response



- http 요청에 응답하는 노드 서버 만들기
 - createServer로 요청 이벤트에 대기
 - req 객체는 요청에 관한 정보가, res 객체는 응답에 관한 정보가 담겨 있음

```
const http = require('http');
```

```
http.createServer((req, res) => {  
  // 여기에 어떻게 응답할지 적습니다.  
});
```

요청과 응답

- res 메서드로 응답 보냄
 - write로 응답 내용을 적고
 - end로 응답 마무리(내용을 넣어도 됨)
- listen(포트) 메서드로 특정 포트에 연결

```
const http = require('http');

http.createServer((req, res) => {
  res.writeHead(200, { 'Content-Type': 'text/html; charset=utf-8' });
  res.write('<h1>Hello Node!</h1>');
  res.end('<p>Hello Server!</p>');
})

.listen(8080, () => { // 서버 연결
  console.log('8080번 포트에서 서버 대기 중입니다!');
});
```

요청과 응답

- 스크립트를 실행하면 8080 포트에 연결됨

콘솔

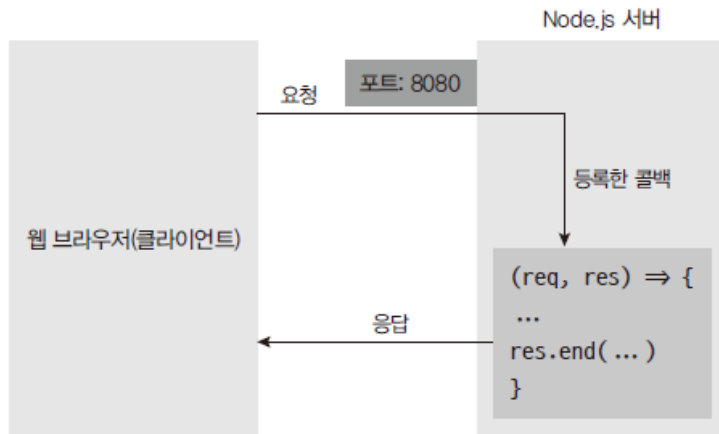
```
$ node server1
```

```
8080번 포트에서 서버 대기 중입니다!
```

- localhost:8080 또는 http://127.0.0.1:8080에 접속

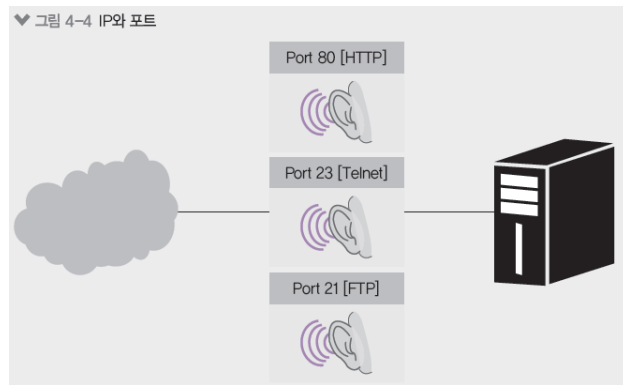
Hello Node!

Hello Server!



요청과 응답

- localhost는 컴퓨터 내부 주소
 - 외부에서는 접근 불가능
- 포트는 서버 내에서 프로세스를 구분하는 번호
 - 기본적으로 http 서버는 80번 포트 사용(생략가능, https는 443)
 - 예) www.gilbut.com:80 -> www.github.com
 - 다른 포트로 데이터베이스나 다른 서버 동시에 연결 가능



요청과 응답

- 이벤트 리스너 추가 가능
 - listening 과 error 이벤트 가능

server1-1.js

```
const http = require('http');

const server = http.createServer((req, res) => {
  res.writeHead(200, { 'Content-Type': 'text/html; charset=utf-8' });
  res.write('<h1>Hello Node!</h1>');
  res.end('<p>Hello Server!</p>');
});
server.listen(8080);

server.on('listening', () => {
  console.log('8080번 포트에서 서버 대기 중입니다!');
});
server.on('error', (error) => {
  console.error(error);
});
```

요청과 응답

- `createServer` 를 여러 번 호출하여 한번에 여러 개의 서버 실행 가능
 - 단, 포트를 다르게 지정해야함

```
const http = require('http');

http.createServer((req, res) => {
  res.writeHead(200, { 'Content-Type': 'text/html; charset=utf-8' });
  res.write('<h1>Hello Node!</h1>');
  res.end('<p>Hello Server!</p>');
})

.listen(8080, () => { // 서버 연결
  console.log('8080번 포트에서 서버 대기 중입니다!');
});

http.createServer((req, res) => {
  res.writeHead(200, { 'Content-Type': 'text/html; charset=utf-8' });
  res.write('<h1>Hello Node!</h1>');
  res.end('<p>Hello Server!</p>');
})

.listen(8081, () => { // 서버 연결
  console.log('8081번 포트에서 서버 대기 중입니다!');
});
```

요청과 응답

- 문자열 대신 html 파일의 형태로 서비스
 - fs 모듈

server2.html

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="utf-8" />
  <title>Node.js 웹 서버</title>
</head>
<body>
  <h1>Node.js 웹 서버</h1>
  <p>만들 준비되셨나요?</p>
</body>
</html>
```

Synchronous : 요청을 보낸 후 해당 요청의 응답을 받아야 다음 동작을 실행
Asynchronous : 요청을 보낸 후 응답과 관계없이 다음 동작을 실행할 수 있는 방식

비동기 처리 (파일 처리)

server2.js

```
const http = require('http');
const fs = require('fs').promises;

http.createServer(async (req, res) => {
  try {
    const data = await fs.readFile('./server2.html');
    res.writeHead(200, { 'Content-Type': 'text/html; charset=utf-8' });
    res.end(data);
  } catch (err) {
    console.error(err);
    res.writeHead(500, { 'Content-Type': 'text/plain; charset=utf-8' });
    res.end(err.message);
  }
})

.listen(8081, () => {
  console.log('8081번 포트에서 서버 대기 중입니다!');
});
```

프로미스의 필요성

- Callback Hell
 - 콜백 함수를 익명 함수로 전달하는 과정이 반복되어 코드의 들여쓰기 수준이 감당하기 힘들 정도로 깊어지는 현상
 - 이벤트 처리 및 서버 통신과 같은 비동기적인 작업을 수행 시 발생
 - 코드 가독성 저하
 - `setTimeout(function, ms, parameter1, parameter2, ...)`
 - 왜 아래처럼 하면 안되나요
 - `setTimeout(function(parameter1, parameter2), ms)`

Espresso
Espresso, Americano
Espresso, Americano, Mocha
Espresso, Americano, Mocha, Latte

```

setTimeout(
  (name) => {
    let coffeeList = name;
    console.log(coffeeList);

    setTimeout(
      (name) => {
        coffeeList += ', ' + name;
        console.log(coffeeList);

        setTimeout(
          (name) => {
            coffeeList += ', ' + name;
            console.log(coffeeList);

            setTimeout(
              (name) => {
                coffeeList += ', ' + name;
                console.log(coffeeList);
              },
              500,
              'Latte',
            );
          },
          500,
          'Mocha',
        );
      },
      500,
      'Americano',
    );
  },
  500,
  'Espresso',
);

```

프로미스의 필요성

- 해결책 1
 - 익명함수 대신 기명함수
 - 함수 갯수 증가
 - 함수 이름을 따라다녀야함

```
let coffeeList = '';  
  
const addEspresso = (name) => {  
  coffeeList = name;  
  console.log(coffeeList);  
  setTimeout(addAmericano, 500, 'Americano');  
};  
  
const addAmericano = (name) => {  
  coffeeList += ', ' + name;  
  console.log(coffeeList);  
  setTimeout(addMocha, 500, 'Mocha');  
};  
  
const addMocha = (name) => {  
  coffeeList += ', ' + name;  
  console.log(coffeeList);  
  setTimeout(addLatte, 500, 'Latte');  
};  
  
const addLatte = (name) => {  
  coffeeList += ', ' + name;  
  console.log(coffeeList);  
};  
  
setTimeout(addEspresso, 500, 'Espresso');
```

프로미스의 필요성

- 해결책 2
 - Promise : 내용이 실행은 되었지만 결과를 아직 반환하지 않은 객체
 - Promise 내부에 특정 함수(resolve, reject) 를 실행해야만
그 다음 코드(then 또는 catch)로 넘어감
 - Resolve(성공리턴값) -> then으로 연결
 - Reject(실패리턴값) -> catch로 연결
 - Finally 부분은 무조건 실행됨
 - 비동기 작업이 완료될 때 resolve 또는 reject 를 호출함

```
const condition = true; // true면 resolve, false면 reject
const promise = new Promise((resolve, reject) => {
  if (condition) {
    resolve('성공');
  } else {
    reject('실패');
  }
});
// 다른 코드가 들어갈 수 있음

promise
  .then((message) => {
    console.log(message); // 성공(resolve)한 경우 실행
  })
  .catch((error) => {
    console.error(error); // 실패(reject)한 경우 실행
  })
  .finally(() => { // 끝나고 무조건 실행
    console.log('무조건');
  });
```

프로미스의 필요성

- 여전히 긴 느낌

```
new Promise((resolve) => {
  setTimeout(() => {
    let name = 'Espresso';
    console.log(name);
    resolve(name);
  }, 500);
})
.then((prevName) => {
  return new Promise((resolve) => {
    setTimeout(() => {
      let name = prevName + ', Americano';
      console.log(name);
      resolve(name);
    }, 500);
  });
})
```

```
.then((prevName) => {
  return new Promise((resolve) => {
    setTimeout(() => {
      let name = prevName + ', Mocha';
      console.log(name);
      resolve(name);
    }, 500);
  });
})
.then((prevName) => {
  return new Promise((resolve) => {
    setTimeout(() => {
      let name = prevName + ', Latte';
      console.log(name);
      resolve(name);
    }, 500);
  });
})
```

프로미스의 필요성

- 해결책 3 : Promise + Async/await
 - <https://ko.javascript.info/async-await>
 - async 는 function 앞에 위치
 - 해당 function 은 항상 promise 를 반환하게 됨
 - await 는 async function 안에서만 동작함
 - await 키워드는 promise 가 처리될 때까지 기다림
 - 결과는 그 이후에 반환
 - promise.then 과 동일 (하지만 더 직관적)

```
async function f() {  
  
  let promise = new Promise((resolve, reject) => {  
    setTimeout(() => resolve("완료!"), 1000)  
  });  
  
  let result = await promise; // 프라미스가 이행될 때까지 기다림 (*)  
  
  alert(result); // "완료!"  
}  
  
f();
```


프로미스의 필요성

- 해결책 3 : Promise + Async/await

```
const addCoffee = (name) => {
  return new Promise((resolve) => {
    setTimeout(() => {
      resolve(name);
    }, 500);
  });
};

const coffeeMaker = async () => {
  let coffeeList = '';
  let _addCoffee = async (name) => {
    coffeeList += (coffeeList ? ', ' : '') + (await addCoffee(name));
  };
  await _addCoffee('Espresso');
  console.log(coffeeList);
  await _addCoffee('Americano');
  console.log(coffeeList);
  await _addCoffee('Mocha');
  console.log(coffeeList);
  await _addCoffee('Latte');
  console.log(coffeeList);
};

coffeeMaker();
```

예외처리

- 예외 (Exception) : 처리하지 못한 에러
 - 노드 스레드를 멈춤
 - 노드는 기본적으로 싱글 스레드라 스레드가 멈춘다는 것은 프로세스가 멈추는 것
 - 에러 처리는 필수
 - 기본적으로 try catch 로 예외를 처리
 - 에러가 생겨도 스레드가 멈추지 않는다는 것

```
setInterval(() => {  
  console.log('시작');  
  try {  
    throw new Error('서버를 고장내주마!');  
  } catch (err) {  
    console.error(err);  
  }  
}, 1000);
```

예외처리

- 비동기 메서드
 - 따로 처리하지 않아도 됨
 - 콜백 함수에서 에러 객체 제공
- 프로미스
 - 자동으로 catch 함
 - throw된 에러를 자동으로 rejected 상태로 변환
 - 다만 상황에 맞게 try catch 를 써야함
 - 예를 들어, 로그인 실패 시, 왜 실패하였는지에 대한 처리가 필요할 때

error2.js

```
const fs = require('fs');

setInterval(() => {
  fs.unlink('./abcdefg.js', (err) => {
    if (err) {
      console.error(err);
    }
  });
}, 1000);
```

error3.js

```
const fs = require('fs').promises;

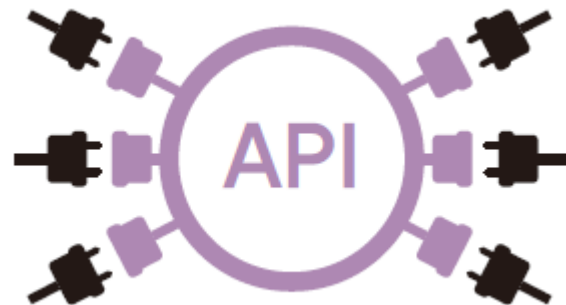
setInterval(() => {
  fs.unlink('./abcdefg.js')
}, 1000);
```

Contents

- NodeJS 소개
- 호출 스택과 이벤트 루프
- 요청과 응답
- [REST API](#)

Rest API

- API: Application Programming Interface
 - 다른 애플리케이션에서 현재 프로그램의 기능을 사용할 수 있게 함
 - 웹 API: 다른 웹 서비스의 기능을 사용하거나 자원을 가져올 수 있게 함
- 서버에 요청을 보낼 때는 주소를 통해 요청의 내용을 표현
 - /index.html이면 index.html을 보내달라는 뜻
 - 항상 html을 요구할 필요는 없음
 - 서버가 이해하기 쉬운 주소가 좋음

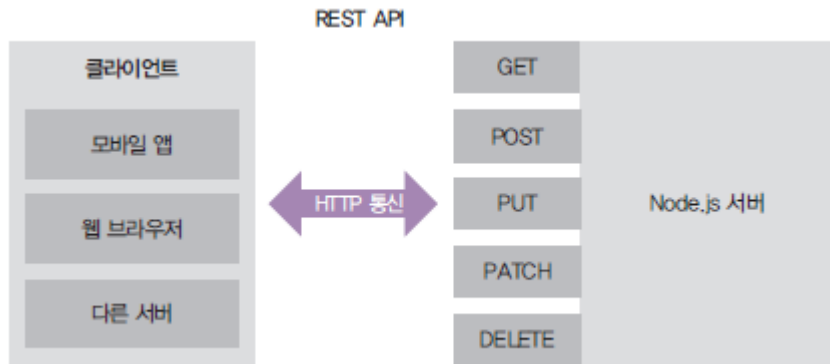


Rest API

- REST API(Representational State Transfer)
 - 서버의 자원을 정의하고 자원에 대한 주소를 지정하는 방법
 - /user이면 사용자 정보에 관한 정보를 요청하는 것
 - /post면 게시물에 관련된 자원을 요청하는 것
- HTTP 요청 메서드
 - GET: 서버 자원을 **가져오려고** 할 때 사용
 - POST: 서버에 자원을 **새로 등록하고자** 할 때 사용(또는 뭘 써야할 지 애매할 때)
 - PUT: 서버의 자원을 요청에 들어있는 자원으로 **치환하고자할 때** 사용
 - PATCH: 서버 자원의 **일부만 수정하고자 할 때** 사용
 - DELETE: 서버의 자원을 **삭제하고자할 때** 사용

Rest API

- 클라이언트가 누구든 서버와 HTTP 프로토콜로 소통 가능
 - iOS, 안드로이드, 웹이 모두 같은 주소로 요청 보낼 수 있음
 - 서버와 클라이언트의 분리



RESTful

- RESTful
 - REST API를 사용한 주소 체계를 이용하는 서버
 - GET /user는 사용자를 조회하는 요청, POST /user는 사용자를 등록하는 요청

HTTP 메서드	주소	역할
GET	/	restFront.html 파일 제공
GET	/about	about.html 파일 제공
GET	/users	사용자 목록 제공
GET	기타	기타 정적 파일 제공
POST	/users	사용자 등록
PUT	/users/사용자id	해당 id의 사용자 수정
DELETE	/users/사용자id	해당 id의 사용자 제거

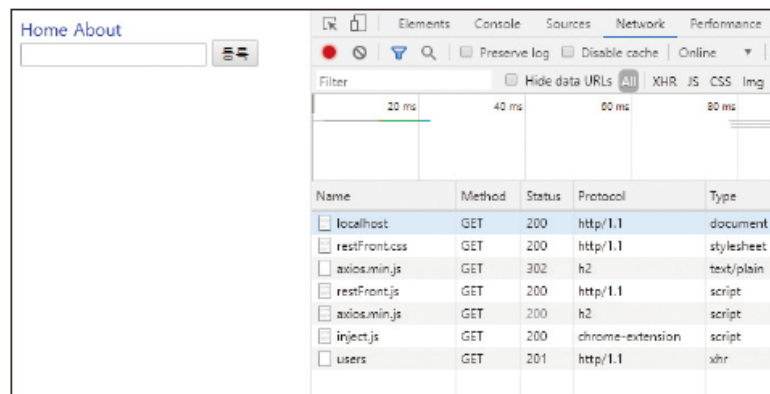
REST 서버 만들기

- <https://github.com/ZeroCho/nodejs-book/tree/master/ch4/4.2>
- 소스코드 참조 (restServer.js)
 - GET method
 - /, /about : HTML 파일 읽어서 전송
 - /users : user 데이터를 전송함 (JSON 형태)
 - POST, PUT method
 - 클라이언트로부터 데이터를 받는 부분
 - DELETE method
 - 주소에 들어있는 키 사용자를 제거
 - 해당 주소 없을 시 404 NOT FOUND error 응답

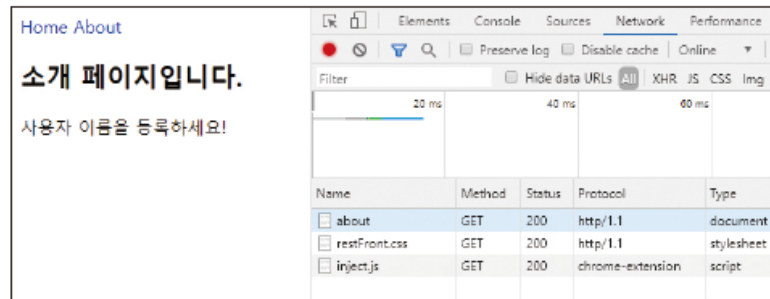
REST 서버 실행

- localhost:8082 접속
 - node restServer

♥ 그림 4-7 Home 클릭 시



♥ 그림 4-8 About 클릭 시



REST 요청의 확인

- 크롬 개발자 도구 - Network 탭
 - 요청 내용을 실시간 확인 가능
 - Name은 요청 주소, Method는 요청 메서드, Status는 HTTP 응답 코드
 - Protocol은 HTTP 프로토콜, Type은 요청 종류(xhr은 AJAX 요청)

Name	Method	Status	Protocol	Type
localhost	GET	200	http/1.1	document
restFront.css	GET	200	http/1.1	stylesheet
axios.min.js	GET	302	h2	text/plain
restFront.js	GET	200	http/1.1	script
axios.min.js	GET	200	h2	script
inject.js	GET	200	chrome-extension	script
users	GET	201	http/1.1	xhr
user	POST	201	http/1.1	xhr

Front-end 와 Back-end 의 통신

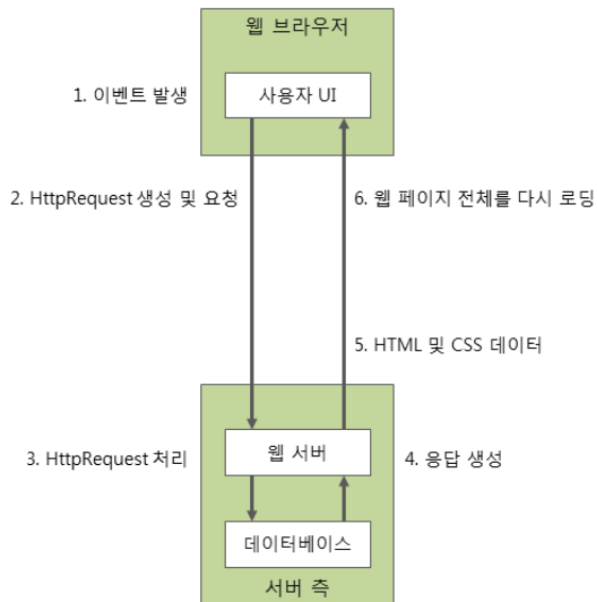
- Front 와 Back 이 주고 받을 수 있어야 함
 - Back (express) 에서는 CRUD method 를 만들고 대기함
 - Front 는 Back 에서 설계한대로 요청을 넣음
 - How?
 - Front 에서의 비동기 통신 방법을 이해해야할 필요

Front-end 에서 구현하는 비동기 통신

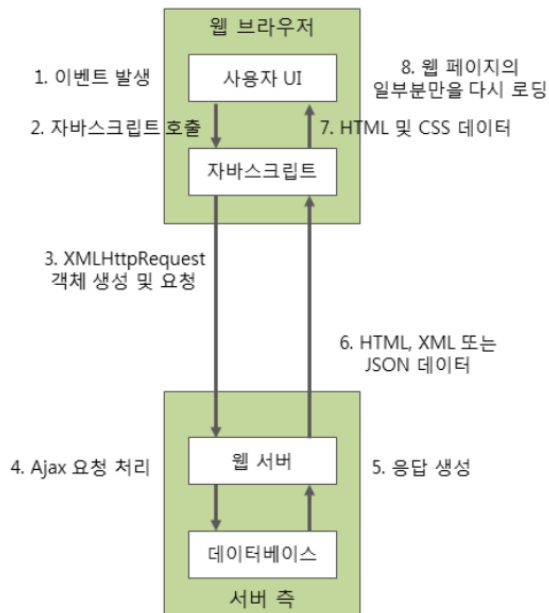
• AJAX : Asynchronous JavaScript and XML

- Ajax는 웹 페이지 전체를 다시 로딩 (새로고침)하지 않고도, 웹 페이지의 일부분만을 갱신 가능
- Ajax를 이용하면 백그라운드 영역에서 서버와 통신하여, 그 결과를 웹 페이지의 일부분에만 표시 가능

< 기존 웹 응용 프로그램의 동작 원리 >



< Ajax를 이용한 웹 응용 프로그램의 동작 원리 >



Front-end 에서 구현하는 비동기 통신

- 서버로 요청을 보내는 코드
 - 라이브러리 없이는 브라우저가 지원하는 XMLHttpRequest 객체 이용
 - AJAX 요청 시 Axios 라이브러리 활용
 - HTML에 아래 스크립트를 추가하면 사용할 수 있음.

```
<script src="https://unpkg.com/axios/dist/axios.min.js"></script>
<script>
  // 여기에 예제 코드를 넣으세요.
</script>
```

Front-end 에서 구현하는 비동기 통신

- GET 요청 보내기
 - axios.get 함수의 인수로 요청을 보낼 주소를 넣으면 됨
 - 프로미스 기반 코드 = async/await 사용 가능

```
axios.get('https://www.zerocho.com/api/get')
  .then((result) => {
    console.log(result);
    console.log(result.data); // {}
  })
  .catch((error) => {
    console.error(error);
  });
```

```
(async () => {
  try {
    const result = await axios.get('https://www.zerocho.com/api/get');
    console.log(result);
    console.log(result.data); // {}
  } catch (error) {
```

Front-end 에서 구현하는 비동기 통신

- POST 요청을 하는 코드(데이터를 담아 서버로 보내는 경우)
 - 전체적인 구조는 비슷하나 두 번째 인수로 데이터를 넣어 보냄

```
(async () => {  
  try {  
    const result = await axios.post('https://www.zerocho.com/api/post/json', {  
      name: 'zerocho',  
      birth: 1994,  
    });  
    console.log(result);  
    console.log(result.data); // {}  
  } catch (error) {  
    console.error(error);  
  }  
})();
```


Thank you

Q & A